EE 148a [Winter 2026]

Problem Set 4

Due: 03/10/26 at 11:59 p.m.
Submit: on gradescope
Originally Prepared By: Firdavs Nasriddinov and Sophia Stiles

Collaboration is allowed, just list the names of the people you collaborated in the header. If you choose to use LaTeX to type your submission, we recommend filling in this template.

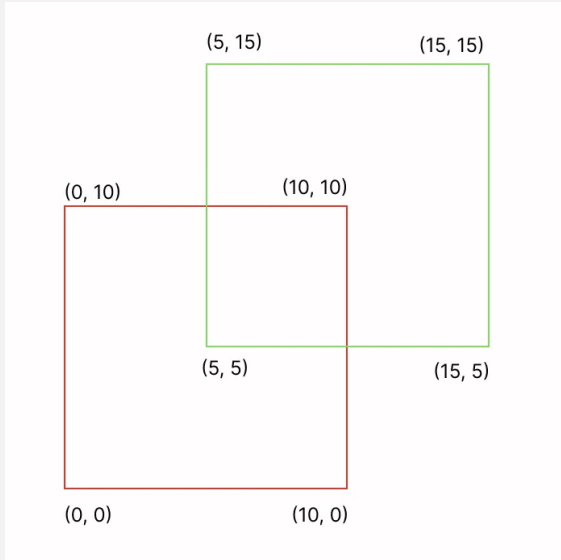
You do not need to upload your code as part of the submission. However, the coding problems can be a useful resource for your projects.

If you have any other questions, come to office hours or ask us on Piazza for a quick response.

Problem 1: Data Annotation **[2 pts]**

(a) Conceptual

i) The Intersection Over Union (IoU) metric evaluates how well a model's predicted bounding box aligns with the actual bounding box of an object in an input image. In the image below, the predicted box is shown in red, and the ground truth box is in green. Calculate the IoU of these two bounding boxes and provide a numerical answer. (2 points)



ii) (a few phrases) What does an IoU of 0 signify in terms of bounding box similarity? What about an IoU of 1?

Now, let's walk through a data annotation workflow using a tool called CVAT.ai.

1. Download the ZIP file here which contains 6 unannotated images.
2. Follow the tutorial guide here (starting at 0:56) to set up a CVAT task and label your data with bounding boxes. Your CVAT task should only use the labels "sign" and "tree".
3. When you are finished labeling:
   - Click "Menu" on the top left.
   - Export the job dataset and click "OK." This should take you to the requests dashboard.
   - At the dashboard, click on the export job you just created and wait for it to display 100%.
   - Click on the 3 dots to the right of that export job and select "Download." This will download a ZIP file containing your XML annotations.
   - Attach screenshots of your .xml file here.

(b) Annotation Instructions (0 points)

Computer vision models often depend on human-annotated images, which can be obtained through crowdsourcing platforms like Amazon Mechanical Turk. Clear, detailed labeling instructions can improve standardization in your training data.

i) Write a set of instructions detailing how you labeled your data in **part (a)** and include them here. Share your instructions with a classmate, have them annotate the dataset using your instructions, and obtain the .xml file of their annotations.

ii) Implement the TODOs in this Colab notebook and include a link to your Colab copy here. Also, paste in the outputs generated by running the final cell of your notebook.

(c) Summary (0 points)

What was the average IOU among the matches found between your bounding box annotations and your partner's? What edge cases or nuances did your instructions fail to address?

Solution

Student solution here.

Problem 2: Supervision **[2 pts]**

(a) Unsupervised Learning: Autoencoders (0 points)

Autoencoders are neural networks that learn lower-dimensional latent representations of input data in a fully-unsupervised manner. They're useful for data compression and noise reduction.

i) Make a copy of this Colab and complete all TODOs under the "Autoencoder Network Architecture" section of the notebook.

ii) Now, we will perform some ablation studies on the autoencoder. Complete all TODOs under the "Ablations" section. Include the loss plots and test accuracy metrics for both ablation studies.

iii) Link your Colab copy here. List 3 interesting observations from your ablation results in **part ii** and include a hypothesis for each observation.

(b) Self-Supervised Learning: Contrastive Learning (0 points)

Self-supervised learning allows a model to generate labels from unlabeled data. Contrastive learning, a subset of self-supervised learning, trains models by comparing data pairs, maximizing similarity for positive pairs and minimizing it for negative ones. This question explores SimCLR, a popular contrastive learning framework. The original paper is linked here.

Make a copy of this Colab and implement all TODOs. Experiment with the augmentation transforms to achieve at least 80% accuracy after 10 training epochs.

(c) Reflection (2 points)

In a few sentences, describe when you might want to use one supervision approach over the other.

Solution

Student solution here.

Problem 3: Variational Autoencoders **[20 pts]**

Variational autoencoders (VAEs) learn probabilistic mappings from latent variables, z, to observed data, x. The goal of a VAE is to approximate the true data distribution, $p^*(x)$, using a parameterized distribution, $p_\theta(x)$, where θ represents the learnable parameters. The original VAE paper is linked here.

(a) Latent Space (5 points)

(1-2 sentences) What distribution is z sampled from?

(b) KL Divergence (5 points)

The VAE optimization objective is typically formulated using Kullback-Leibler (KL) divergence. Specifically, we aim to minimize the divergence $\mathcal{D}_{\mathrm{KL}}(q_\phi(z|x) \parallel p_\theta(z|x))$.

i) (2 sentences) Explain what the terms $q_\phi(z|x)$ and $p_\theta(z|x)$ represent in the divergence formula above.

ii) Name at least one mathematical property of KL-divergence that makes it a good criterion.

(c) The ELBO (5 points)

It turns out that the minimization of KL-divergence is equivalent to the maximization of a formulation called the Evidence Lower Bound (ELBO) — and equivalently, the minimization of the negative of the ELBO:

$$\mathcal{L}_{\theta,\phi}(x) = -\mathbb{E}_{z\sim q_\phi}\left[\log p_\theta(x|z)\right] + \mathcal{D}_{\mathrm{KL}}(q_\phi(z|x) \parallel p_\theta(z))$$

Explain the significance of each term in the ELBO. Give the formal name for both terms and explain the role they play in training a VAE.

Here are a few pointers that may be helpful when implementing VAE loss in **part (e)**:
For the first term, since p_θ is a multivariate Gaussian, we say $p_\theta(x|z) \sim N(f_\theta(z), I)$, where f_θ is the decoding neural network. So,

$$\begin{aligned}
\log p_\theta(x|z) &= \log \frac{1}{I\sqrt{2\pi}} e^{-\frac{1}{2}\left(\frac{x-f_\theta(z)}{I}\right)^2} \\
&\approx \log e^{-(x-f_\theta(z))^2} \\
&= -(x - f_\theta(z))^2 \\
&= -(x - \hat{x}^2)
\end{aligned}$$

For the second term: if we assume q is a neural network with mean μ and standard deviation σ, and that z comes from the distribution found in **part (a)**, we can equivalently express KL divergence as the following:

$$\mathcal{D}_{KL} = \frac{1}{2}(-\log\sigma^2 - 1 + \sigma^2 + \mu^2)$$

A derivation of KL-divergence is linked here for further reading.

(d) Optimizing $\mathcal{L}$ (5 points)

Now that we've defined our objective function $\mathcal{L}$, the next step is to optimize it with respect to the parameters θ and ϕ using stochastic gradient descent (SGD).

The gradient computation with respect to θ is given by:

$$\begin{aligned}
\nabla_\theta \mathcal{L} &= \nabla_\theta \left[-\mathbb{E}_{z\sim q_\phi} \log p_\theta(x|z) + \mathcal{D}_{\mathrm{KL}}(q_\phi(z|x) \parallel p_\theta(z)) \right] \\
&= \nabla_\theta \left[-\mathbb{E}_{z\sim q_\phi} \log p_\theta(x|z) \right] + \nabla_\theta \left[\mathcal{D}_{\mathrm{KL}}(q_\phi(z|x) \parallel p_\theta(z)) \right] \\
&= \nabla_\theta \left[-\mathbb{E}_{z\sim q_\phi} \log p_\theta(x|z) \right] + 0 \\
&= \nabla_\theta \left[-\int q_\phi(z) \log p_\theta(x|z)\, dz \right] \\
&= -\int q_\phi(z) \left[\nabla_\theta \log p_\theta(x|z) \right] dz \\
&= -\mathbb{E}_{z\sim q_\phi} \left[\nabla_\theta \log p_\theta(x|z) \right]
\end{aligned}$$

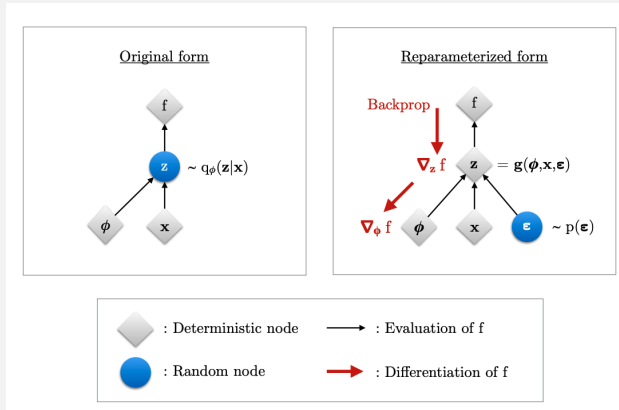
This shows that ∇_θ can be written an expectation, which is useful because we can approximate it using Monte Carlo methods. (For more details, you can refer to Section 2.0.1 of this article, though it's not necessary for this problem set).

Now, follow a similar step-by-step approach from above to compute the gradient w.r.t ϕ for the first term in the loss (note the negative sign in front of the expectation has been removed for simplicity):

$$\nabla_\phi \left[\mathcal{L}_{\text{1st term}} \right] = \nabla_\phi \left[\mathbb{E}_{z\sim q_\phi} \log p_\theta(x|z) \right] = \ ?$$

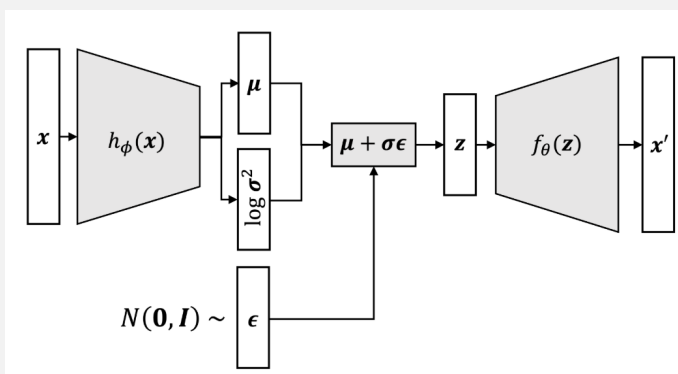
How far can you simplify the gradient? What issue prevents $\nabla_\phi \left[\mathcal{L}_{\text{1st term}} \right]$ from being rewritten directly as an expectation, as we did for $\nabla_\theta \mathcal{L}$ above?

We can apply a simple "reparametrization trick" to fix this problem. The sample $z \sim q_\phi = N(\mu_\phi(x), \Sigma_\phi(x))$ can equivalently be defined as $z = \mu_\phi(x) + \Sigma_\phi(x)^{\frac{1}{2}} \cdot \epsilon$, where $\epsilon \sim N(0, I)$. Take a look at the diagram from the original paper below - notice how z is now deterministic (and consequently differentiable) in the reparametrized form!



(e) Code Implementation (0 points)

Make a copy of the Colab linked here, complete the TODOs, and answer all prompted questions. Refer to the following figure when implementing the model class:



i) Complete defining the VAE model class and loss function. Train the model and upload the printed output.

ii) Complete functions to run model (normal and noisy). Upload a sample normal and noisy run. Determine at around what standard deviation, 'std', the reconstructed image is no longer representative of the original image.

iii) Complete function to interpolate between latent vectors of images of two different digits. Upload a sample run.

iv) Train models for varying KL loss weights. Generate images from randomly sampled latent vectors for each weight value. Upload a sample output. Run all models on sample of test data and compute 2-component PCA of the latent vectors. Upload the generated plot of the PCAs.

v) (3-4 sentences) Briefly explain the effect of varying KL loss weight on 1) randomly generated images and 2) PCAs of latent vectors. (Hint: consider the mean image)

Solution

Student solution here.

Problem 4: Class Imbalance Handling **[0 pts]**

In most real-world data, there exist class imbalances when doing tasks like classification. It is important to account for these class imbalances to ensure the model is able to best learn. We will explore different techniques on a CNN classification task of distinguishing between a cat/dog. Make a copy of a Colab linked here.

(a) Exploratory Data Analysis (0 points)

i) Plot sample images, class distribution, and PCA of images.

ii) Discuss the potential impact of the imbalance on model performance.

(b) Baseline Model (0 points)

i) Complete the function 'train_baseline()' and train a baseline model (using the already defined model class, train functions, and parameters).

ii) Complete the function 'compute_metrics()' and run evaluation on the baseline model. Upload the printed metrics.

iii) (1-2 sentences) Brefly discuss the limitations of accuracy as a performance metric in imbalanced datasets.

iv) (2-4 sentences) Which of the two metrics (AUROC, AUPRC, Precision, Recall, F1) are generally preferred for evaluating model performance on imbalanced datasets? Briefly state why.

(c) Sampling Methods (0 points)

Apply the following resampling methods to handle class imbalance. Complete the methods in the 'Sampler' class first.

i) Implement and evaluate **random majority undersampling**: reduce the number of instances in the majority class. Upload the printed metrics.

ii) Implement and evaluate **random minority oversampling**: increase the number of instances in the minority class. Upload the printed metrics.

iii) Implement and evaluate **Synthetic Minority Over-sampling Technique (SMOTE)**: Generate synthetic samples for the minority class by interpolating between existing minority class samples and their nearest neighbors (use $k = 10$ nearest neighbors). Upload the printed metrics.

iv) (2-3 sentences) Discuss which method is best (by considering the two metrics you mentioned above) for this dataset and hypothesize why the others may not be as effective.

(d) Cost-sensitive Learning (0 points)

Apply cost-sensitive learning to assign a higher misclassification cost to the minority class during training. Intuitively, the class with the lower count should have a higher cost as we want the model to focus on getting that class correct. The loss function we are using (BCEWithLogitsLoss) has a parameter called pos_weight for this.

i) Complete the function 'train_costsensitive_model()' and train the model.

ii) Evaluate and upload the printed metrics.

iii) (2 sentences) Compare the performance with the previous models.

Solution

Student solution here.